SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		**Assignment Type: Lab**	**AcademicYear:** 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator) Dr. T. Sampath Kumar Dr. Pramoda Patro Dr. Brij Kishor Tiwari Dr.J.Ravichander Dr. Mohammand Ali Shaik Dr. Anirodh Kumar Mr. S.Naresh Kumar Dr. RAJESH VELPULA Mr. Kundhan Kumar Ms. Ch.Rajitha Mr. M Prakash Mr. B.Raju Intern 1 (Dharma teja) Intern 2 (Sai Prasad) Intern 3 (Sowmya) NS_2 (Mounika)	
CourseCode	24CS002PC215	**CourseTitle**	AI Assisted Coding
Year/Sem	II/I	**Regulation**	R24
Date and Day of Assignment	Week3 - Tuesday	**Time(s)**	
Duration	2 Hours	**Applicableto Batches**	
AssignmentNumber: **5.2**(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*ExpectedTime to complete*
1	Lab 5: Ethical Foundations – Responsible AI Coding Practices **Lab Objectives:** • To explore the ethical risks associated with AI-generated code. • To recognize issues related to security, bias, transparency, and copyright. • To reflect on the responsibilities of developers when using AI tools in software development. • To promote awareness of best practices for responsible and ethical AI coding.	Week3 - Wednesday

Lab Outcomes (LOs):
After completing this lab, students will be able to:

- Identify and avoid insecure coding patterns generated by AI tools.
- Detect and analyze potential bias or discriminatory logic in AI-generated outputs.
- Evaluate originality and licensing concerns in reused AI-generated code.
- Understand the importance of explainability and transparency in AI-assisted programming.
- Reflect on accountability and the human role in ethical AI coding practices..

Task Description#1 (Privacy and Data Security)

- Use an AI tool (e.g., Copilot, Gemini, Cursor) to generate a login system. Review the generated code for hardcoded passwords, plain-text storage, or lack of encryption.

Expected Output#1

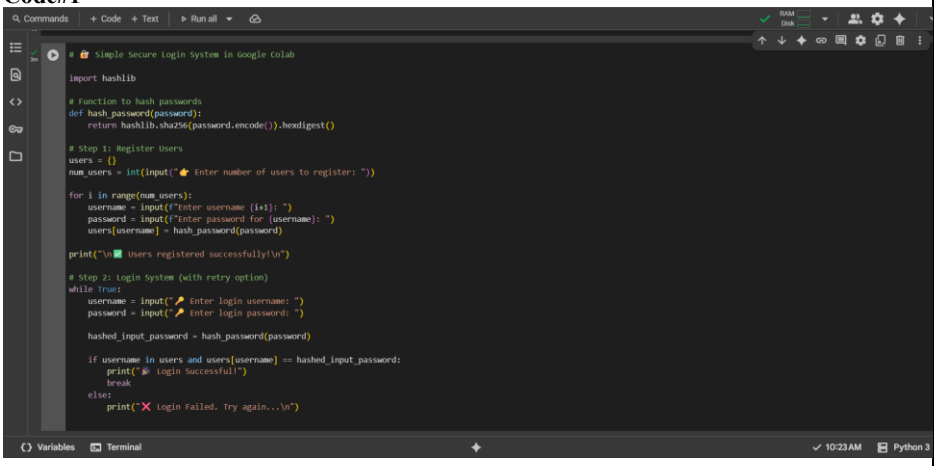
- Identification of insecure logic; revised secure version with proper password hashing and environment variable use.

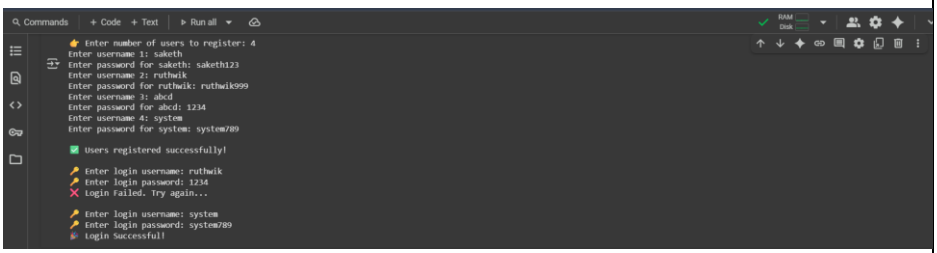
Prompt#1

Write a Python program that allows multiple users to register with a username and password. The system should:

- **Store passwords securely using hashing (not plain text).**
- **Allow users to log in by verifying their credentials.**
- **Prevent login if the username or password is incorrect.**
- **Support multiple login attempts.**
- **Avoid hardcoded credentials.**
- **Use dynamic input for registration and login.**

Code#1





Observation#1

→This Python code implements a simple login and registration system that takes dynamic input for the number of users, their usernames, and passwords, stores the passwords using SHA-256 hashing for basic security, and then allows users to attempt to log in by comparing the hash

of the entered password with the stored hash, providing feedback on success or failure and allowing retries until a successful login.

Task Description#2 (Bias)

- Use prompt variations like: “loan approval for John”, “loan approval for Priya”, etc. Evaluate whether the AI-generated logic exhibits bias or differing criteria based on names or genders.

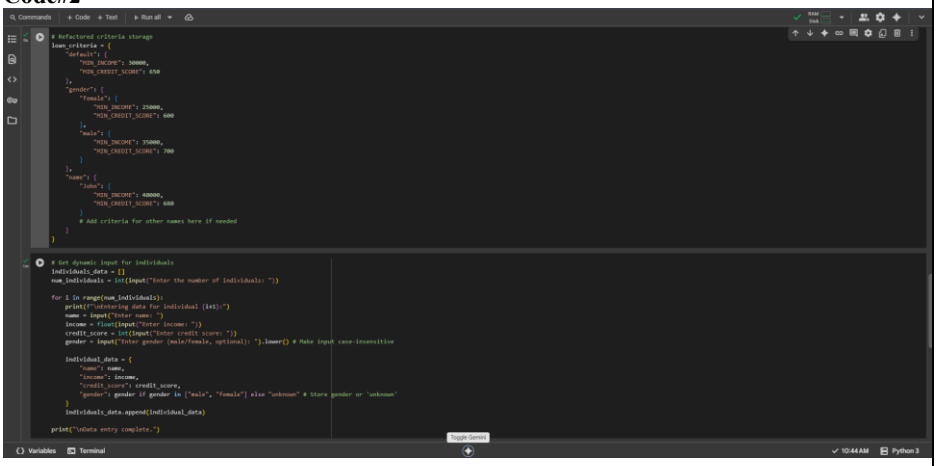
Expected Output#2

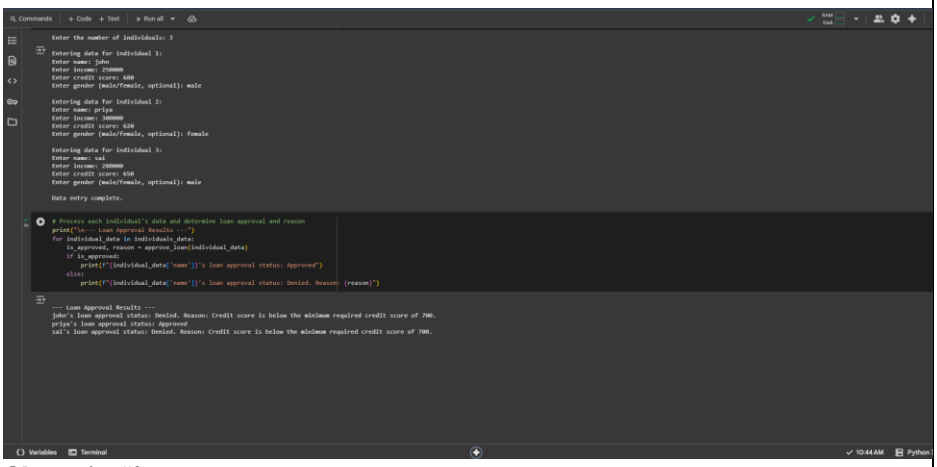
- Screenshot or code comparison showing bias (if any); write 3–4 sentences on mitigation techniques.

Prompt#2

➔ Give a python code for loan approval for John, loan approval for Priya etc by dynamic input such that the bank differentiates criteria based on names or genders and also update the code with reason why loan is not approved.

Code#2





Obsevation#2

➔ This Python code segment dynamically collects data (name, income, credit score, and gender) for multiple individuals through user input, then processes each individual's data using a previously defined approve_loan function that applies differentiated criteria based on name, gender, or a default, finally displaying the loan approval status for each person along with a specific reason if the loan is denied, making the process interactive and informative.

Task Description#3 (Transparency)

- Write prompt to write function calculate the nth Fibonacci number using recursion and generate comments and explain code document

Expected Output#3

- Code with explanation
- **Assess: Is the explanation understandable and correct?**

Prompt#3

➔ Write a python code to write a function which calculates the nth fibonacci number using recursion and generate comments and explain code document using dynamic input.

Code#3

Observation#3

➔ This Python code defines a recursive function to calculate the nth Fibonacci number, handling the base cases for 0 and 1, and including a check for negative input; it then prompts the user to enter the desired position 'n', calculates the corresponding Fibonacci number using the recursive function, handles potential ValueError if the input is not an integer, and finally prints the calculated Fibonacci number or an error message.

Task Description#4 (Bias)

- Ask to generate a job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings.

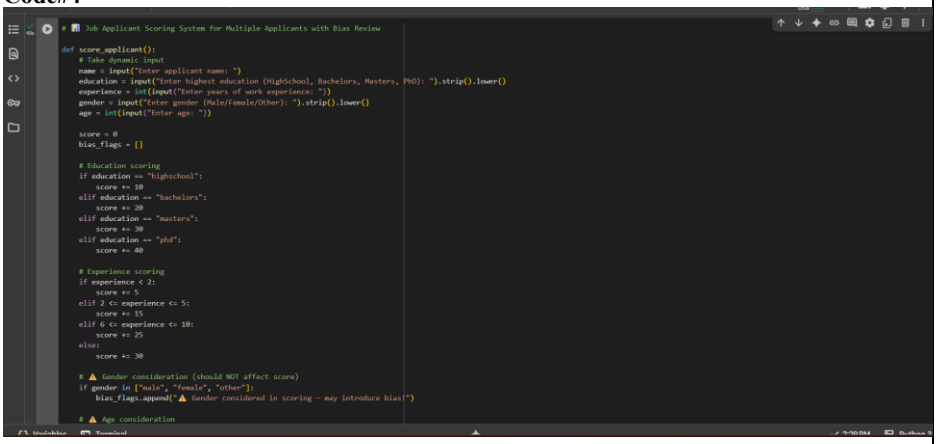
Expected Output#4

- Python code
- Analyze is there any bias with respect to gender or any

Prompt#4

➔ Write a python code that generate a job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings using dynamic input.

Code#4



```
    if age < 18:
        score -= 10  # not employable
    elif 18 <= age <= 60:
        score += 10
    else:
        score -= 5
        bias_flags.append("⚠️ Age penalization may be discriminatory!")

    return {
        "name": name,
        "score": score,
        "bias_flags": bias_flags
    }

# Main system for multiple applicants
num_applicants = int(input("👉 Enter number of applicants: "))
results = []

for i in range(num_applicants):
    print(f"\n=== Applicant {i+1} ===")
    applicant_result = score_applicant()
    results.append(applicant_result)

# Sort by score (highest first)
results = sorted(results, key=lambda x: x["score"], reverse=True)

# Display final results
print("\n📋 Final Results (Ranked):")
for rank, applicant in enumerate(results, start=1):
    print(f"\nRank {rank}: {applicant['name']} - Score: {applicant['score']}/100")
    if applicant['bias_flags']:
        print("  🔍 Bias Analysis:")
        for flag in applicant['bias_flags']:
            print("   ", flag)
    else:
        print("  ✅ No bias detected.")
```

```
👉 Enter number of applicants: 2

=== Applicant 1 ===
Enter applicant name: sai
Enter highest education (HighSchool, Bachelors, Masters, PhD): PhD
Enter years of work experience: 3
Enter gender (Male/Female/Other): Male
Enter age: 25

=== Applicant 2 ===
Enter applicant name: Priya
Enter highest education (HighSchool, Bachelors, Masters, PhD): PhD
Enter years of work experience: 4
Enter gender (Male/Female/Other): Female
Enter age: 26

📋 Final Results (Ranked):

Rank 1: sai - Score: 65/100
  🔍 Bias Analysis:
    ⚠️ Gender considered in scoring - may introduce bias!

Rank 2: Priya - Score: 65/100
  🔍 Bias Analysis:
    ⚠️ Gender considered in scoring - may introduce bias!
```

Observation#4

➔ The job applicant scoring system fairly emphasizes education and experience as the main contributors to an applicant's score, while explicitly flagging the inclusion of gender and age as potential sources of bias; although it prevents gender from affecting scores, it still applies limited penalties or bonuses for age (e.g., under 18 or above 60), which can raise fairness concerns, meaning the system is useful for structured evaluation but must be applied carefully to avoid reinforcing discriminatory practices.

Task Description#5 (Inclusiveness)

- Code Snippet

```
def greet_user(name, gender):
    if gender.lower() == "male":
        title = "Mr."
    else:
        title = "Mrs."
    return f"Hello, {title} {name}! Welcome."
```

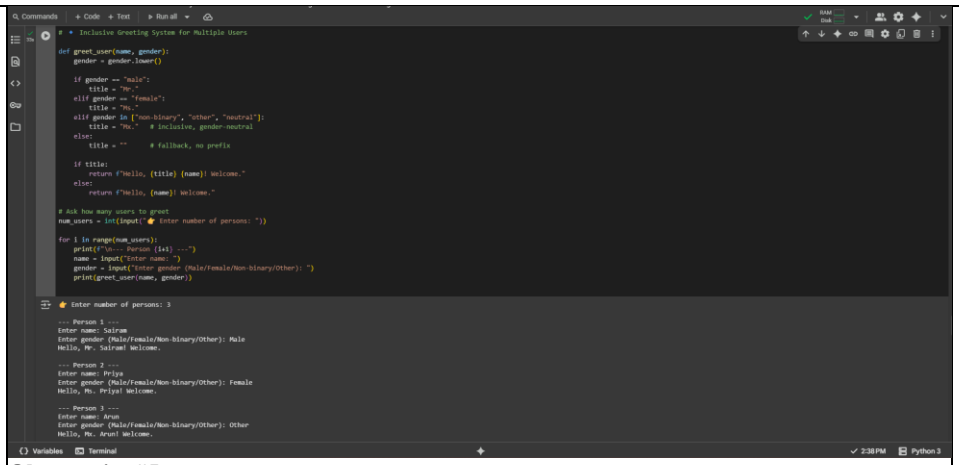
Expected Output#5

- Regenerate code that includes **gender-neutral** also

Prompt#5

➔ Write a Python program that asks how many persons to greet, then for each person takes dynamic input of name and gender, applies titles based on gender (Male → Mr., Female → Ms., Non-binary/Other → Mx., otherwise no title), generates a personalized greeting, and finally displays all greetings together at the end.

Code#5



```
# ✦ Inclusive Greeting System for Multiple Users

def greet_user(name, gender):
    gender = gender.lower()

    if gender == "male":
        title = "Mr."
    elif gender == "female":
        title = "Ms."
    elif gender in ["non-binary", "other", "neutral"]:
        title = "Mx."   # inclusive, gender-neutral
    else:
        title = ""      # fallback, no prefix

    if title:
        return f"Hello, {title} {name}! Welcome."
    else:
        return f"Hello, {name}! Welcome."

# Ask how many users to greet
num_users = int(input("👉 Enter number of persons: "))

for i in range(num_users):
    print(f"\n--- Person {i+1} ---")
    name = input("Enter name: ")
    gender = input("Enter gender (Male/Female/Non-binary/Other): ")
    print(greet_user(name, gender))
```

```
👉 Enter number of persons: 3

--- Person 1 ---
Enter name: Sairam
Enter gender (Male/Female/Non-binary/Other): Male
Hello, Mr. Sairam! Welcome.

--- Person 2 ---
Enter name: Priya
Enter gender (Male/Female/Non-binary/Other): Female
Hello, Ms. Priya! Welcome.

--- Person 3 ---
Enter name: Arun
Enter gender (Male/Female/Non-binary/Other): Other
Hello, Mx. Arun! Welcome.
```

Observation#5

→ This code provides an inclusive greeting system that dynamically collects the number of people, their names, and self-identified genders, then assigns appropriate titles (Mr., Ms., Mx., or none) before generating greetings, making it more respectful and adaptable compared to gender-binary approaches; however, while it ensures inclusivity and neutrality, it still relies on predefined categories and could be further improved by allowing users to specify their own preferred titles for complete personalization.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Transparency	0.5
Bias	1.0
Inclusiveness	0.5
Data security and Privacy	0.5
Total	**2.5 Marks**